

What is GPT and How Does It Work?

Quick Interview / Viva Notes — A simple but technically accurate explanation of GPT, tokenization, embeddings, positional information, self-attention, multi-head attention, the decoder, and detokenization.

1. What is GPT?

GPT stands for **Generative Pre-trained Transformer**. It is a type of large language model (LLM) based on the Transformer architecture. GPT models are designed to understand context and generate text by predicting the next token based on the tokens that came before it.

Important correction: GPT was developed by **OpenAI**, not Google. The Transformer architecture was introduced by Google researchers in the 2017 paper *Attention Is All You Need*. GPT uses the Transformer architecture, specifically a decoder-style architecture.

2. High-Level Flow of GPT

Input Text	→	Tokenization	→	Token IDs
Token IDs	→	Embeddings	→	Positional Information
Representations	→	Self-Attention	→	Multi-Head Attention
Contextual Representation	→	Decoder Layers	→	Next-Token Prediction
Predicted Tokens	→	Detokenization	→	Generated Text

3. Step 1 — Tokenization

Before GPT can process text, the text is split into smaller units called **tokens**. A token can be a complete word, part of a word, punctuation, or sometimes another useful text unit. Tokens are not necessarily individual characters.

Example: the sentence "**Dogs are running**" may be represented by several tokens. The exact tokenization depends on the tokenizer used by the model.

Key point: Tokenization converts human-readable text into token IDs that a neural network can process.

4. Step 2 — Token Embeddings

Each token ID is mapped to a numerical vector called a **token embedding**. An embedding is a dense numerical representation learned by the model. During training, tokens that occur in related contexts can develop useful relationships in the embedding space.

General definition: Vector embeddings are numerical representations of data that allow machine-learning systems to represent and compare semantic or contextual relationships efficiently.

5. Step 3 — Positional Information

A Transformer processes tokens in parallel, so it needs information about **where each token occurs** in the sequence. Positional information helps the model distinguish different word orders.

For example, "**The dog chased the cat**" and "**The cat chased the dog**" contain similar tokens but have different meanings because their positions and relationships are different.

Note: In modern Transformer models, positional information can be implemented in different ways, including learned positional embeddings or other positional encoding/rotation techniques.

6. Why RNNs Were Not Enough

Earlier sequence models such as **RNNs** processed sequences step by step. This could make it difficult to efficiently handle very long-range dependencies.

For example, in the phrase "**I deposited money in the bank**", the context tells us that "bank" refers to a financial institution. In "**We sat near the river bank**", "bank" refers to the land beside a river.

Transformers address this using **self-attention**, allowing each token to consider other relevant tokens in the sequence when building its contextual representation.

7. Step 4 — Self-Attention

Self-attention allows a token to look at other tokens in the same input sequence and determine which ones are important for understanding its meaning in context.

For example, in "**The dog was sleeping because it was tired**", attention can help the model relate "**it**" to the relevant earlier word "**dog**".

A simplified explanation is that the Transformer creates three representations for each token: **Query (Q)**, **Key (K)**, and **Value (V)**. Attention scores are calculated from Q and K, and those scores determine how much information is taken from the V representations.

Simplified formula: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$

8. Step 5 — Multi-Head Attention

Instead of calculating attention only once, **multi-head attention** uses multiple attention heads. Different heads can learn different relationships or patterns in the same sequence.

Example: "**The dog was in the train. The dog was sleeping. The dog was brown in color.**" Different attention heads may learn relationships involving the subject, actions, descriptions, or other contextual dependencies.

Important: Multi-head attention does not simply mean "more attention." It provides multiple learned attention patterns that are combined to create a richer contextual representation.

9. Step 6 — Transformer Decoder

GPT is based on a **decoder-only Transformer**. The decoder contains repeated layers that include masked self-attention, feed-forward neural networks, residual connections, and normalization.

The attention is **causal/masked**, meaning that when predicting the next token, the model cannot look at future tokens. It can only use the tokens already available.

For example, if the input is "**The sky is**", the model calculates probabilities for possible next tokens such as "blue", "clear", or other candidates.

10. Step 7 — Next-Token Prediction

The final decoder representation is passed through a language-model output layer to produce a probability distribution over the model's vocabulary. The model then selects or samples a next token according to the decoding strategy.

This process repeats: the newly generated token becomes part of the context, and the model predicts the next token again. Repeating this process produces a complete response.

11. Step 8 — Detokenization

After token IDs have been generated, the tokenizer converts those tokens back into readable text. This process is called **detokenization**.

So the simplified complete pipeline is: **Text** → **Tokens** → **Token IDs** → **Embeddings** → **Positional Information** → **Attention/Transformer Layers** → **Next-Token Probabilities** → **Generated Token IDs** → **Detokenization** → **Text**.

12. Best Short Interview Answer

“GPT stands for Generative Pre-trained Transformer. It is a decoder-only Transformer-based language model. First, input text is tokenized into tokens and token IDs. These IDs are converted into embeddings, and positional information is added so the model understands token order. The Transformer then uses masked self-attention and multi-head attention to understand relationships between tokens. The model predicts the next token based on the previous context, repeatedly generating tokens until the response is complete. Finally, the generated tokens are detokenized into human-readable text.”

13. Important Corrections to Remember

Incorrect / Confusing	Correct Concept
GPT = Gen Pretrained Transformer	GPT = Generative Pre-trained Transformer
Every character gets exactly one token	Text is split into tokens; a token can be a word, subword, punctuation, etc.
GPT was designed by Google	GPT was developed by OpenAI; the original Transformer architecture was introduced by Google
Positional embedding tells the matching word	Positional information tells the model about token order/position.
Self-attention is only for matching words	Self-attention learns contextual relationships and assigns different importance to tokens.
After attention, it directly detokenizes	The model first produces next-token probabilities and generates token IDs; detokenization happens at the end.

One-line memory trick: Tokenize → Embed → Add Position → Attend → Predict Next Token → Repeat → Detokenize.